

DSA Contest 3: Math, Two pointers, backtracking, Linked list & Trees

* Problems -

1. Children & Rides
2. Special Prime Numbers
3. Flatten binary tree to linked list
4. Permutations → theory lecture

* Children and Rides

Children and Rides

Problem Description

In an amusement park, there are rides being constructed for N children. Each ride has a maximum capacity limit of B for the total weight of the children on board.

You are given an array of weights of children, where $A[i]$ is the weight of the i -th child, and an infinite number of rides. Each ride can carry **at most two** children at the same time, provided the sum of their weights is at most the ride's capacity limit.

Find the minimum number of rides to accommodate all the children.

Problem Constraints

$$1 \leq N \leq 10^5$$

$$0 \leq B \leq 10^9$$

→ 2 children

→ sum of weight $\leq B$

Example Input

Input 1:

$A = [1, 2, 3]$
 $B = 4$

Input 2:

$A = [3, 2, 2, 0, 1]$
 $B = 4$

Example Output

Output 1:

2

Output 2:

3

Example Explanation

Example 1:

We can take elements of $i = 1$ and $i = 3$ and combine them as one ride and the element $i = 2$ can be considered as another ride. So the answer is 2.

Example 2:

We can combine elements of $(i = 2 \text{ and } i = 3)$ and $(i = 1 \text{ and } i = 5)$ and $(i = 4)$ so the answer is 3.

NOTE: In both explanations we have followed 1-based indexing.

Example 2:

We can combine elements of ($i = 2$ and $i = 3$) and ($i = 1$ and $i = 5$) and ($i = 4$) so the answer is 3.

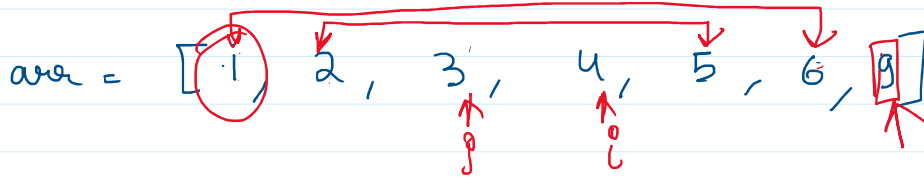
NOTE: In both explanations we have followed 1-based indexing.

$$arr = [2, 1, 5, 9, 3, 4, 6]$$

↓
weights

max cap. of ride
↑
 $B = 9$

sort the array



$$1 + 9 > B$$

$$1 + 6 \leq B$$

$$5 + 2 \leq B$$

$$3 + 4 \leq B$$

$$rides = 1 + 1 + 1 + 1 = \underline{4 \text{ rides}}$$

$B \geq$ max weight of a
child

* pseudo code -

Arrays.sort(A) → $O(N \log N)$

int i = 0, j = N - 1;

int sides = 0;

while (i <= j) → $O(N)$ Time
{
 int sum = A[i] + A[j];

 if (sum <= B) // sending both in a single side
 {

 sides++;

 i++;

 j--;

 }

 else

 {

 sides++;

 j--;

 }

}

return sides;

TC → $O(N \log N)$
SC → $O(1)$

* Special prime Numbers

Special Prime Numbers

Problem Description

You're a software developer working on a new feature for a financial application. The application provides users with insights into interesting numerical patterns. One of the features involves identifying and analyzing prime numbers with special properties.

Your team lead has assigned you the task of finding prime numbers up to a given number A where the sum of the digits of each prime number is also a prime number. This feature will help users discover unique primes and their properties.

Given an integer A, Your task is to find the number of prime numbers less than or equal to A where the sum of the digits of the prime number is also a prime number.

Problem Constraints

$2 \leq A \leq 10^7$

Input Format

First Argument is an **Integer**.

Output Format

Return an **Integer**, denoting the total number of special prime numbers less than or equal to A.

Example Input

Input 1:

A = 10

Input 2:

A = 30

Example Output

Output 1:

4

Output 2:

7

Example Explanation

Explanation 1:

The prime numbers less than or equal to 10 are [2, 3, 5, 7]. The sum of the digits of the primes are also a prime number.

There are 4 numbers, that satisfy the condition.

Explanation 2:

The prime numbers less than or equal to 30 are [2, 3, 5, 7, 11, 13, 17, 19, 23, 29].

The sum of the digits of the primes are [2, 3, 5, 7, 2, 4, 8, 10, 5, 11].

From the above list, the sum of the digits, which is also prime are [2, 3, 5, 7, 2, 5, 11].

There are 7 numbers, that satisfy the condition.

eg - A = 30

prime numbers $\rightarrow$ [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] $\rightarrow$ keine Sieve

Sum of Digits: 2, 3, 5, 7, 2, 4, 8, 10, 5, 11

Ans $\rightarrow$ 7

A = 30

A = 30
isPrime [31] → 0 to 30 → T F

prime → 19
 sum → 10

Sum of digit (A) ≤ A

eg → 287

$$2 + 8 + 7 \leq \underline{2 \times 100 + 8 \times 10 + 7}$$

isPrime [31]

A = 30

0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17
 F F T T F T F F F F T F T F F F T

18 19 20 21 22 23 24 25 26 27 28 29 30
 F T F F F T F F F F F T F

$$\text{Count} = 1 + 1 + 1 + 1 + 1 + 1 + 1 = 7$$

#code →

boolean isPrime[N+1];

for (i = 2; i ≤ N; i++) isPrime[i] = true;

isPrime[0] = false, isPrime[1] = false;

for (i = 2; i * i ≤ N; i++) {

if (isPrime[i] == true) {

for (j = i * i; j ≤ N; j += i) {

isPrime[j] = false;

}

ans = 0;

for (i = 2; i ≤ N; i++) {

if (isPrime[i] == true && isPrime[sumOfDigits(i)] == true) {
 ans++;

N = 30

O(N * log(N))

prime sieve

OWN

OWN)

```

ans = 0;
for (i = 2; i ≤ N; i++) {
    if (isPrime[i] == true && isPrime[sumOfDigits(i)] == true) {
        ans++;
    }
}
return ans;

```

$T.C \rightarrow O(N \cdot \log N)$
 $S.C \rightarrow O(N)$

```

int sumOfDigits (int x) {
    sum = 0;
    while (x != 0) {
        r = x % 10;
        x = x / 10;
        sum += r;
    }
    return sum;
}

```

$\rightarrow$ no. of digit
 $T.C \rightarrow O(\log_{10} N)$
 $S.C \rightarrow O(1)$

$$r = x \% 10$$

$$\begin{array}{l}
 x = 2 \ 8 \ 7 \ 4 \rightarrow 287 \\
 \text{Sum} = 4 + 7 + 8 + 2 = 21 \downarrow \\
 0 \leftarrow 2 \leftarrow 28
 \end{array}$$

* Flatten Binary Tree to linked list (famous Interview problem)

Flatten Binary Tree to Linked List

Problem Description

Given a binary tree **A**, flatten it to a linked list in-place using preorder traversal. The left child of all nodes should be NULL.

Problem Constraints

1 ≤ size of tree ≤ 100000

Input Format

Input Format

First and only argument is the head of tree A.

Output Format

Return the linked-list after flattening.

Example Input

Input 1:



Input 2:

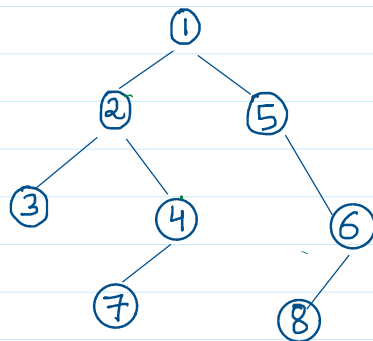


Example Output

Output 1:



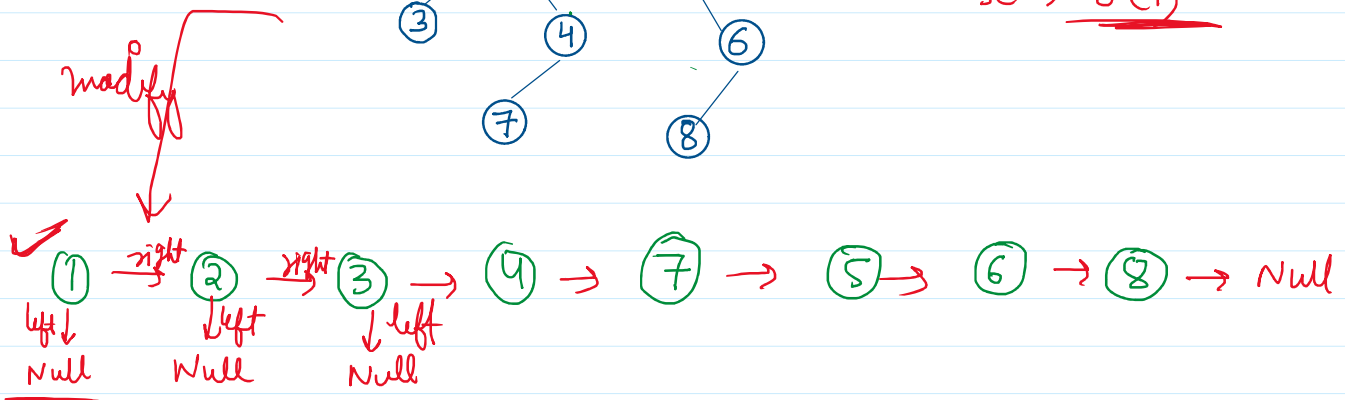
Output 2:

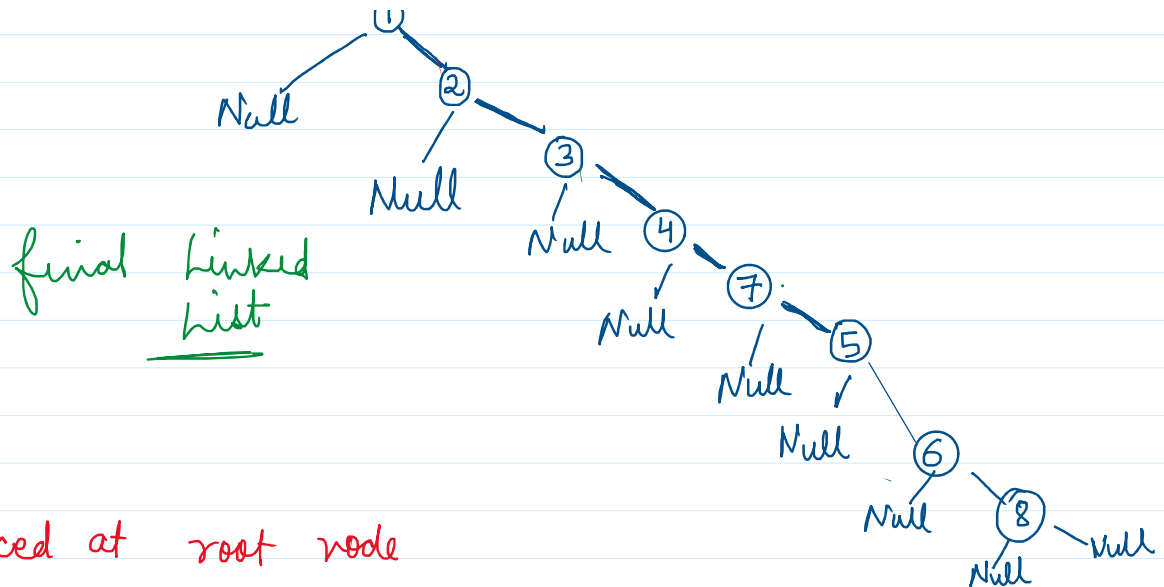


Morris Traversal

TC $\rightarrow O(N)$

SC $\rightarrow O(1)$





Approach →

- ① cur is placed at root node
- ② If left child of cur does not exist, move cur to its right
- ③ If left child exist, then find the rightmost node in the left subtree of cur.
- ④ point the right child of temp to the right child of cur.
- ⑤ point left child of cur as right child now and point left of cur as Null.
- ⑥ repeat from step 1 until cur reaches Null.

pseudo code -

Code :-

Node curr = root;

while (curr != null) {

if (curr.left == null) {

curr = curr.right;

}
else { // If the left is Not Null

Node temp = curr.left;

while (temp.right != null) {

temp = temp.right;

temp.right = curr.right;

curr.right = curr.left;

curr.left = null;

}

return root;

temp becomes right most node
in left subtree of curr

T.C $\rightarrow O(N)$
S.C $\rightarrow O(1)$